University of Geneva

Faculty of Science

Assignment 3

Computational Imaging
14x062

Michel Jean Joseph Donnet

2026-04-04

Contents

1 Introduction

The classical digital photography allows to capture images with a good quality. However, due to the physical properties of the camera, the focus is made on a specific point, and the notion of depth of field appears: objects not focused are blurred.

To respond to this problem, computational imaging allows to refocus the image after it has been captured thanks to analysis of the depth of field. Indeed, the depth of field gives a notion of the distance between the camera and the objects of the scene, and can be estimated by an analysis of the blur of the image, as we will study in the first part of this work. Then, thanks to the depth of field information, a refocus can be performed thanks to application of specific filters according to the depth of field, as we will see in the second part of this work.

To have a more precise estimation of the depth of field, some cameras have been designed like the plenoptic camera. These cameras capture the depth of field by a multi view acquisition system created by a microlens array placed in front of the sensor. The depth of field can then be estimated by an analysis of the multi view acquisition and the refocus can be performed in this way, as we will see in the third part of this work.

2 Task 1: Depth estimation

The code was implemented in matlab. As some random forest was used in matlab, the code wasn't directly reusable in python.

That's why I try to understand the code with the main functions and principles. Then, I used the understanding to implement from scratch a similar pipeline in python using ChatGPT and python libraries like scikit-learn.

The base principle of the depth map estimation is described in the Figure 1 for the training pipeline, and in Figure 2 for the prediction pipeline.

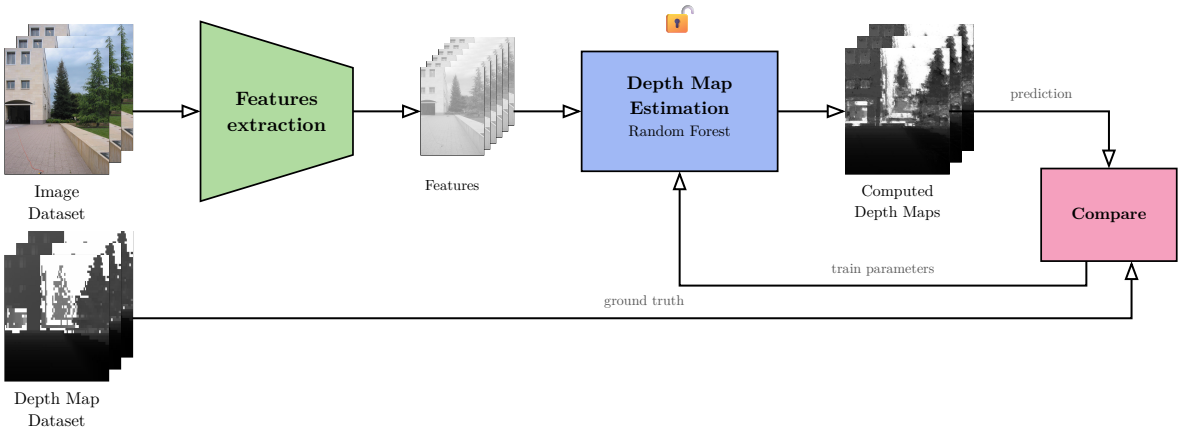


Figure 1: Depth map estimation training based on the given matlab code

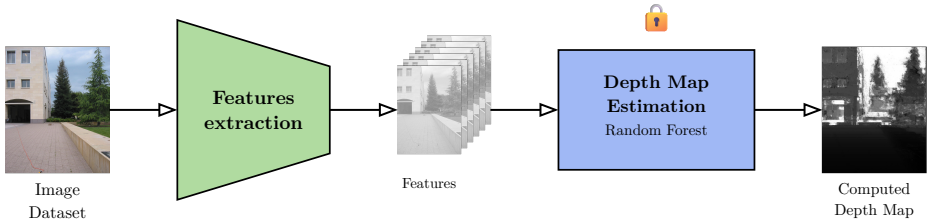


Figure 2: Depth map estimation prediction

2.1 Features

There is a lot of features extracted used to compute the depth map, such as color, edge and texture features.

2.1.1 Color features

As shown on Figure 3, color features are extracted from different color spaces:

- RGB (Red, Green, Blue)
- LUV (Lightness, U and V chrominance channels)
- HSI (Hue, Saturation, Intensity)

They allow to capture different representations of the image that are used in depth estimation. Indeed, an object with a specific color is more likely to be at a specific depth.

2.1.2 Texture features

Texture features are computed by applying laws filters to the image, as shown on Figure 4.

Laws filters are a set of filters designed to capture different levels of texture in an image.

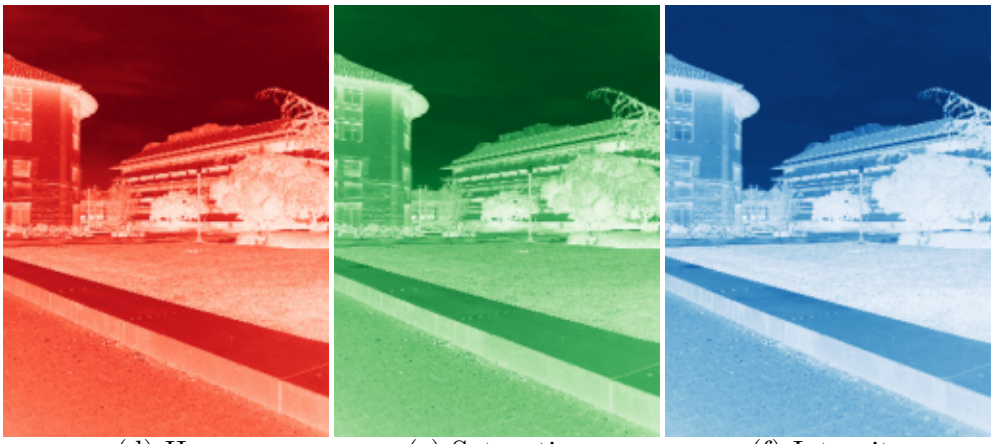
Indeed, some of the filters make a smoothing of the image while others capture fine details.

2.1.3 Edge features

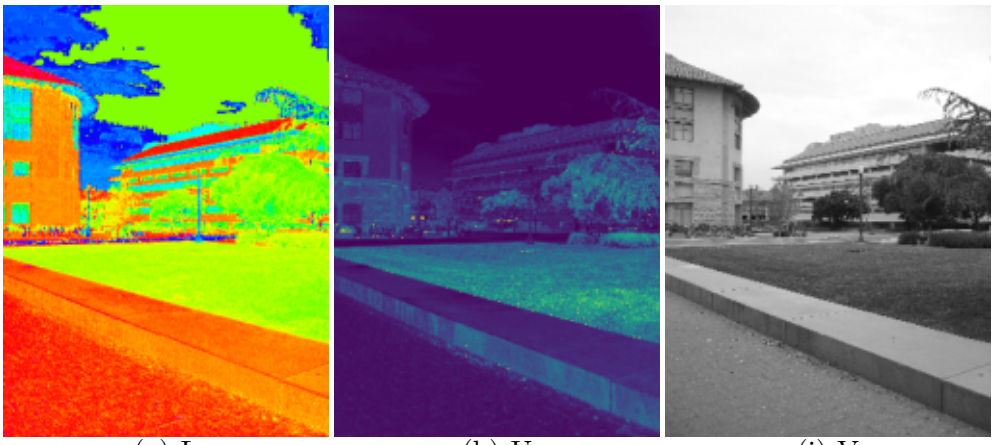
Edge features are computed by applying Navatia-Babu filters to the image, as shown on Figure 5.

Navatia-Babu filters are a set of filters designed to capture edges in an image at different orientations and scales. The current implementation uses 6 filters to capture edges at orientations of 0°, 30°, 60°, 90°, 120° and 150°.

(a) R (b) G (c) B



(d) Hue (e) Saturation (f) Intensity



(g) L (h) U (i) V

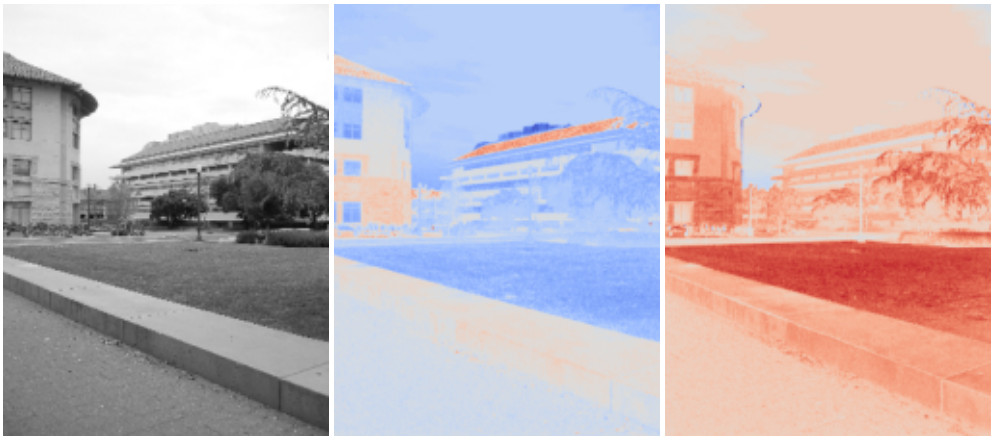
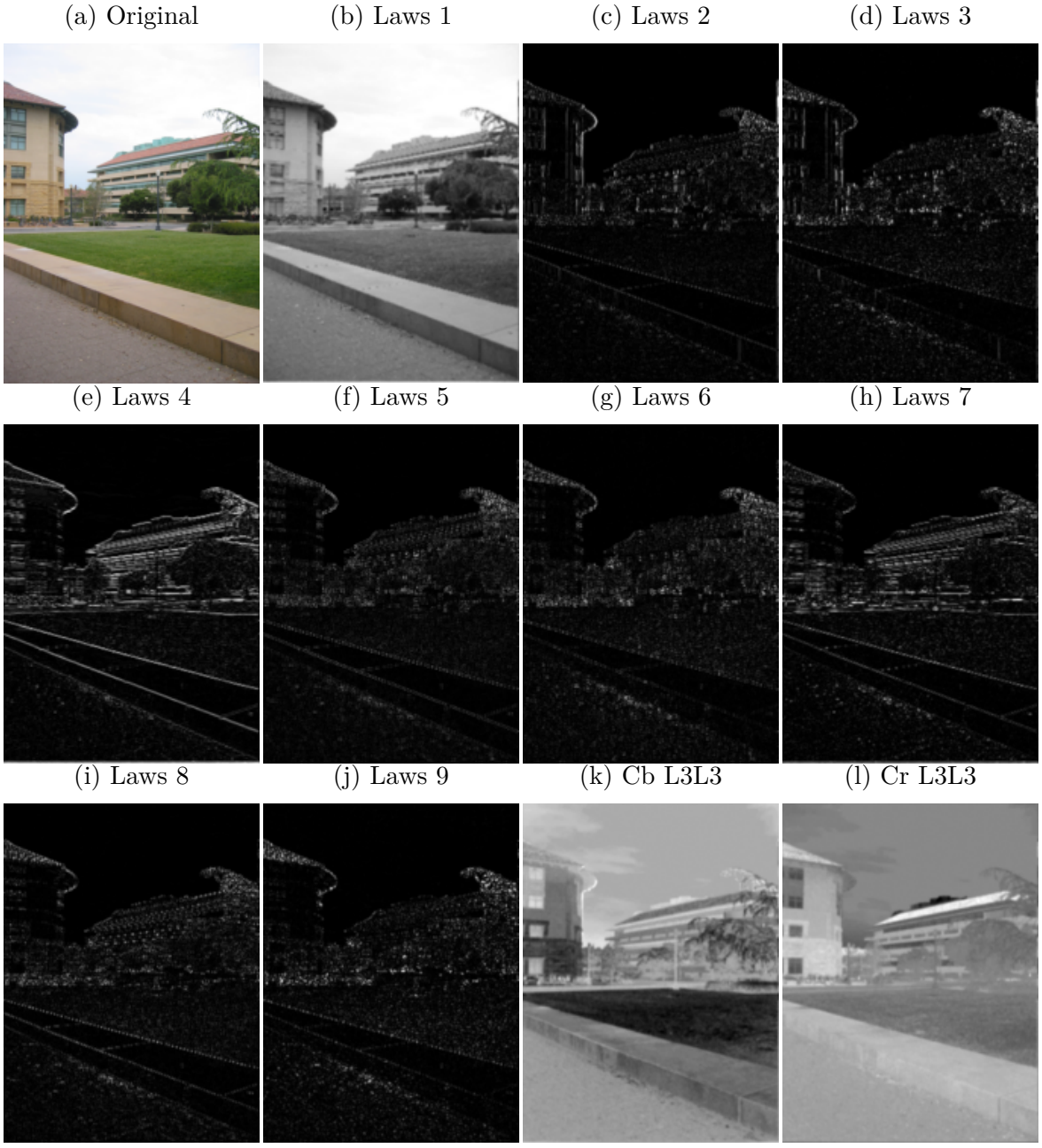


Figure 3: Color features



(a) Original (b) Laws 1 (c) Laws 2 (d) Laws 3

(e) Laws 4 (f) Laws 5 (g) Laws 6 (h) Laws 7

(i) Laws 8 (j) Laws 9 (k) Cb L3L3 (l) Cr L3L3

Figure 4: Laws texture features

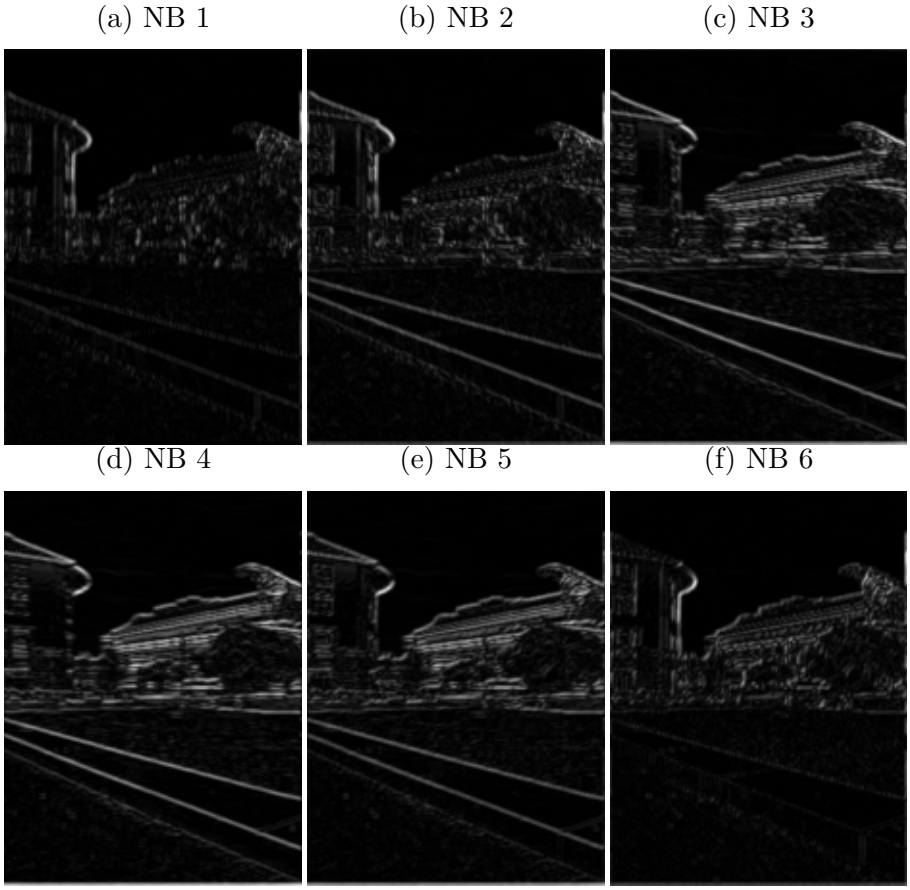


Figure 5: Navatia-Babu edge features

2.2 Results

The results obtained with the implemented pipeline are shown on Figure 6.

We can see that the predicted depth map captures the general depth of the scene, even if it is not perfect and noisy. Indeed, the depth of the sky is correctly estimated to be far while the depth of the ground is correctly estimated to be close.

However, as we can see, the depth is quite noisy and not very accurate, which is due to the single capture of the image and the complexity of the depth estimation problem that requires a lot of information to be accurate, like for instance a focal stack or a focus-aperture stack or a multi view acquisition as in plenoptic cameras.

(a) Input image

(b) Predicted depth

Figure 6: Depth estimation results

3 Task 2: Image refocusing

In this task, we will use the depth map of an image or some sub-aperture images to perform some refocusing of the image, based on an existing implementation in matlab. To convert the matlab code to python, I used another approach than in task 1: this time, I converted the matlab code to python line by line.

Then, I used Github Copilot to update the documentation to numpy style and to debug the code to make it properly working in python.

3.1 Refocusing using depth map

For the refocusing using depth map, the principle is to select the pixels of the image that are at a specific depth an apply a blur filter that increases with the distance to the selected depth on other pixels, as shown on Figure 7.

The obtained result is a refocused image at the selected depth, as shown on Figure 8. Thanks to the depth map, the refocusing is accurate and the result is quite good. Indeed, the information of the depth is crucial to perform a good refocusing, as it allows to apply the right amount of blur on each pixel according to its depth to have a realistic refocus that follows the physical properties of the camera.

So thanks to the depth map, a good refocusing can be performed after the capture of the image, which can be only optained thanks to computational imaging techniques.

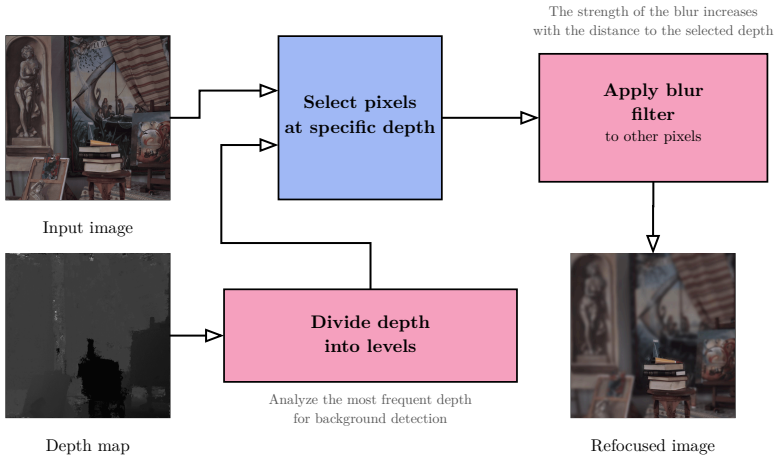


Figure 7: Refocusing using depth map

(a) Input image (b) Depth map (c) Refocused image

Figure 8: Refocusing on the books using depth map

3.2 Refocusing using sub-aperture images

4 Task 3: Image refocusing and depth estimation

5 Implementation

6 Results

7 Discussion

8 Conclusion